

DSC630 4.2 Assignment Ayden Trusty

January 18, 2026

1 4.2 Assignment: R/Python Refresher

1.1 DSC630 Ayden Trusty

===== */

Title: Assignment 1.2

Author(s): Abbott, Dean. Applied Predictive Analytics (Textbook)

Date: 5 December 2025

Modified By: Ayden Trusty

Description: This program is to refresh my understanding/practice of using Python and/or R.

```
[1]: import pandas as pd
```

```
df = pd.read_csv("als_data.csv")
df.head()
```

```
[1]:
```

	ID	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	\
0	1	65	57.0	40.5	38.0	0.066202	
1	2	48	45.0	41.0	39.0	0.010453	
2	3	38	50.0	47.0	45.0	0.008929	
3	4	63	47.0	44.0	41.0	0.012111	
4	5	63	47.0	45.5	42.0	0.008292	

	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_median	ALSFRS_Total_min	...	\
0	-0.965608	30	28.0	22	...	
1	-0.921717	37	33.0	21	...	
2	-0.914787	24	14.0	10	...	
3	-0.598361	30	29.0	24	...	
4	-0.444039	32	27.5	20	...	

	Sodium_min	Sodium_range	SubjectID	trunk_max	trunk_median	trunk_min	\
0	143.0	0.017422	533	8	7.0	7	
1	136.0	0.010453	649	8	7.0	5	
2	140.0	0.008929	1234	5	0.0	0	
3	138.0	0.012469	2492	5	5.0	3	
4	138.0	0.008292	2956	6	4.0	1	

	trunk_range	Urine.Ph_max	Urine.Ph_median	Urine.Ph_min
0	0.002646	6.0	6.0	6.0
1	0.005386	7.0	5.0	5.0
2	0.008929	6.0	5.0	5.0
3	0.004988	7.0	6.0	5.0
4	0.008489	6.0	5.0	5.0

[5 rows x 101 columns]

1.1.1 1. Remove any data that is not relevant to the patient's ALS condition.

After looking through the dataset, I don't think that patient identifiers are relevant since they're strictly labels, not really anything to do with their ALS condition.

```
[3]: cols_to_drop = [c for c in ["ID", "SubjectID"] if c in df.columns]
X = df.drop(columns=cols_to_drop)

print("Dropped:", cols_to_drop)
print("Shape after drop:", X.shape)

df.head()
```

Dropped: ['ID', 'SubjectID']

Shape after drop: (2223, 99)

```
[3]:
```

	ID	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	\
0	1	65	57.0	40.5	38.0	0.066202	
1	2	48	45.0	41.0	39.0	0.010453	
2	3	38	50.0	47.0	45.0	0.008929	
3	4	63	47.0	44.0	41.0	0.012111	
4	5	63	47.0	45.5	42.0	0.008292	

	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_median	ALSFRS_Total_min	...	\
0	-0.965608	30	28.0	22	...	
1	-0.921717	37	33.0	21	...	
2	-0.914787	24	14.0	10	...	
3	-0.598361	30	29.0	24	...	
4	-0.444039	32	27.5	20	...	

	Sodium_min	Sodium_range	SubjectID	trunk_max	trunk_median	trunk_min	\
0	143.0	0.017422	533	8	7.0	7	
1	136.0	0.010453	649	8	7.0	5	
2	140.0	0.008929	1234	5	0.0	0	
3	138.0	0.012469	2492	5	5.0	3	
4	138.0	0.008292	2956	6	4.0	1	

	trunk_range	Urine.Ph_max	Urine.Ph_median	Urine.Ph_min
0	0.002646	6.0	6.0	6.0

1	0.005386	7.0	5.0	5.0
2	0.008929	6.0	5.0	5.0
3	0.004988	7.0	6.0	5.0
4	0.008489	6.0	5.0	5.0

[5 rows x 101 columns]

```
[4]: print("Non-numeric cols:", X.select_dtypes(exclude="number").columns.tolist())
      print("Total missing values:", X.isna().sum().sum())
```

```
Non-numeric cols: []
Total missing values: 0
```

1.1.2 2. Apply a standard scalar to the data.

```
[5]: from sklearn.preprocessing import StandardScaler

      scaler = StandardScaler()
      X_scaled = scaler.fit_transform(X)
```

1.1.3 3. Create a plot of the cluster silhouette score versus the number of clusters in a K-means cluster.

```
[14]: # I was looking into the messages as to why I was getting some of the errors,
      ↪even though the code ran cleanly. I'm going to try running this first and
      ↪see if it can help.

      import os
      os.environ["OMP_NUM_THREADS"] = "4"
      os.environ["LOKY_MAX_CPU_COUNT"] = "4"
```

```
[17]: import numpy as np
      import matplotlib.pyplot as plt
      from sklearn.cluster import KMeans
      from sklearn.metrics import silhouette_score

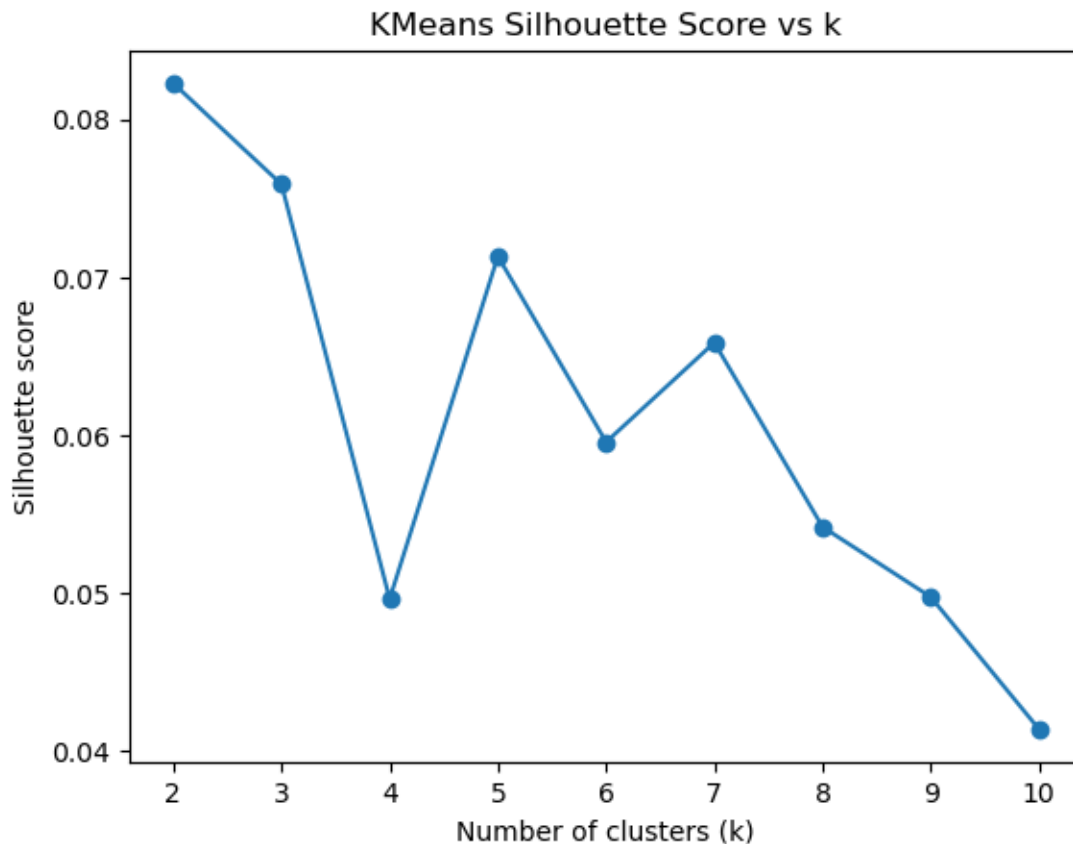
      rng = np.random.default_rng(42)
      sample_size = 1000
      idx = rng.choice(X_scaled.shape[0], size=sample_size, replace=False)
      X_samp = X_scaled[idx]

      ks = range(2, 11)
      sil_scores = []

      for k in ks:
          km = KMeans(n_clusters=k, random_state=42, n_init=5, max_iter=300)
          labels_samp = km.fit_predict(X_samp)
          sil_scores.append(silhouette_score(X_samp, labels_samp))
```

```
plt.figure()
plt.plot(list(ks), sil_scores, marker="o")
plt.xticks(list(ks))
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette score")
plt.title("KMeans Silhouette Score vs k")
plt.show()

best_k = list(ks)[int(np.argmax(sil_scores))]
best_score = float(np.max(sil_scores))
print("Best k:", best_k, "with silhouette:", best_score)
```



Best k: 2 with silhouette: 0.08229801764395882

Still showing the same errors, but at least the plot is displaying correctly. I just don't like how it looks when there's a bunch of pink message blocks.

```
[16]: # I guess there's always this approach.
import warnings
warnings.filterwarnings("ignore", message="Could not find the number of_
↳physical cores*")
```

```
warnings.filterwarnings("ignore", message="KMeans is known to have a memory_↵
↵leak on Windows*")
```

That finally worked, but I would have rather had the first approach work for me since that's what the environment was displaying as an option. That's not the first time jupyter lab/notebook has done that to me.

1.1.4 4. Use the plot created in (3) to choose an optimal number of clusters for K-means. Justify your choice.

The silhouette score plot indicates that $k = 2$ yields the highest silhouette score (0.082). While the overall silhouette values are low, this is expected for complex clinical data with overlapping patient characteristics. Increasing the number of clusters resulted in lower silhouette scores, indicating reduced cluster cohesion and separation.

1.1.5 5. Fit a K-means model to the data with the optimal number of clusters chosen in part (4).

```
[18]: k_opt = best_k

kmeans = KMeans(n_clusters=k_opt, random_state=42, n_init=10, max_iter=300)
clusters = kmeans.fit_predict(X_scaled)

# Add cluster labels back onto a copy of the original feature table (unscaled)
results = X.copy()
results["cluster"] = clusters

results["cluster"].value_counts()
```

```
[18]: cluster
1    1112
0    1111
Name: count, dtype: int64
```

1.1.6 6. Fit a PCA transformation with two features to the scaled data.

```
[19]: from sklearn.decomposition import PCA

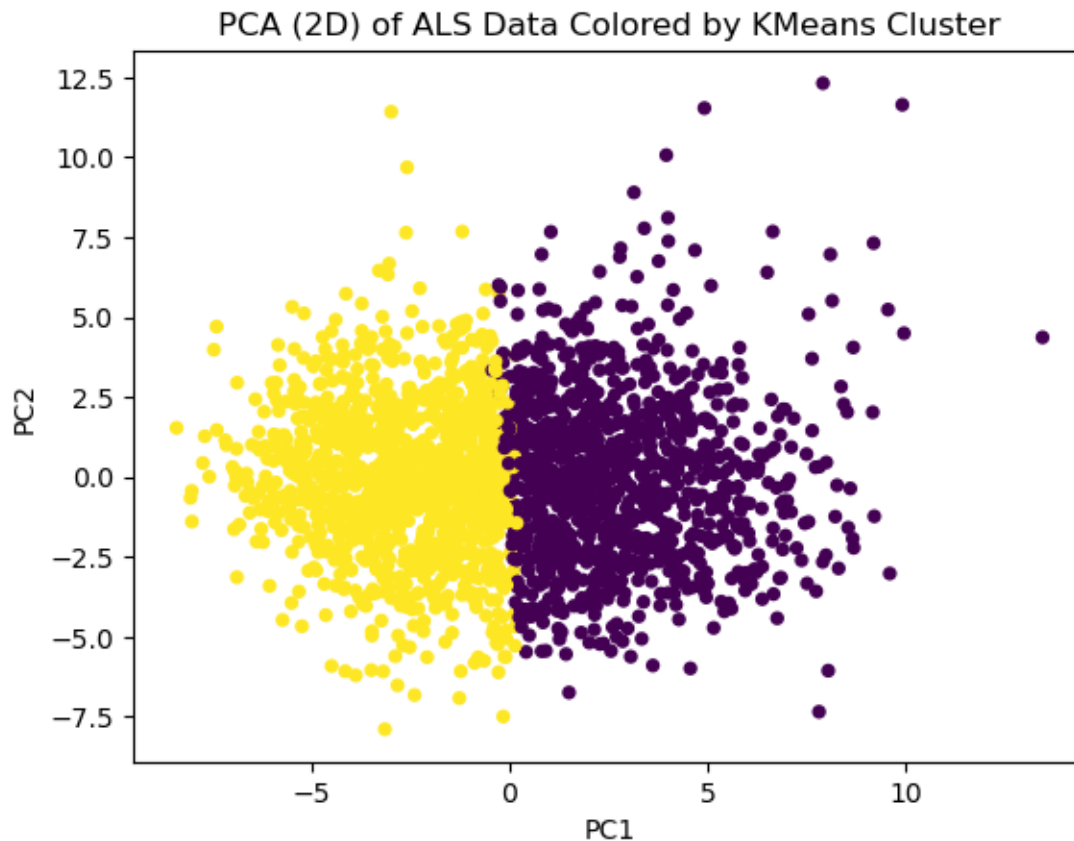
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total explained variance (2 PCs):", pca.explained_variance_ratio_.sum())
```

```
Explained variance ratio: [0.11330548 0.06446615]
Total explained variance (2 PCs): 0.17777162549973646
```

1.1.7 7. Make a scatterplot the PCA transformed data coloring each point by its cluster value.

```
[20]: plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, s=18)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA (2D) of ALS Data Colored by KMeans Cluster")
plt.show()
```



1.1.8 8. Summarize your results and make a conclusion.

```
[21]: als_cols = [c for c in results.columns if "ALSFRS" in c] + [
    c for c in ["respiratory_median", "trunk_median", "Age_mean"] if c in
    ↪ results.columns
]

summary = results.groupby("cluster")[als_cols].agg(["mean", "median", "count"])
summary
```

```

[21]:
      ALSFRS_slope      ALSFRS_Total_max      \
            mean      median count      mean median count
cluster
0      -1.027138 -0.950521  1111      28.597660  29.0  1111
1      -0.429679 -0.384414  1112      34.784173  35.0  1112

      ALSFRS_Total_median      ALSFRS_Total_min ... \
            mean median count      mean ...
cluster
0      22.410666  23.0  1111      13.133213 ...
1      31.794964  32.0  1112      26.615108 ...

      ALSFRS_Total_range respiratory_median      trunk_median \
            count      mean median count      mean
cluster
0      1111      3.396040  3.5  1111      3.555806
1      1112      3.789119  4.0  1112      6.229766

      Age_mean
            median count      mean median count
cluster
0      3.5  1111  54.469847  55.0  1111
1      6.0  1112  54.630396  55.0  1112

[2 rows x 24 columns]

```

After running the silhouette analysis, I chose $k = 2$ as the number of clusters since it produced the highest silhouette score (0.082). The score itself is fairly low, but that was not unexpected given the complexity of ALS data and the amount of overlap that likely exists between patients. When I increased the number of clusters beyond two, the silhouette scores dropped, which suggested that adding more clusters was not improving the structure of the model.

Once I fit the K-means model with two clusters, I found that the groups were almost perfectly balanced, with 1111 patients in one cluster and 1112 in the other. Looking at ALSFRS-related measures helped make sense of what the clusters represented. One cluster showed a much more negative ALSFRS slope (-1.03) along with lower ALSFRS total scores, which points to faster functional decline and more severe disease progression. The other cluster had a less negative slope (-0.43) and consistently higher functional scores, suggesting slower progression.

I then used PCA to reduce the data to two dimensions for visualization. The first two principal components explained about 17.8% of the total variance, which explains why the PCA scatterplot showed noticeable overlap between clusters. This overlap aligns with the low silhouette score and highlights the continuous nature of ALS progression rather than clearly separated patient groups. Overall, even though the clustering was not strongly separated, the results still provided useful exploratory insight into

different ALS progression patterns.